

Simulating JAK-STAT signaling with BioNetGen



Laura F. Strube, PhD

University of Pittsburgh, Medical School
Department of Computational and Systems Biology

SIAM Life Sciences, Cleveland
08 July 2026

Simulating JAK-STAT signaling with BioNetGen

Model Example

- Cytokine-induced STAT protein activation

Constructing a BioNetGen Model

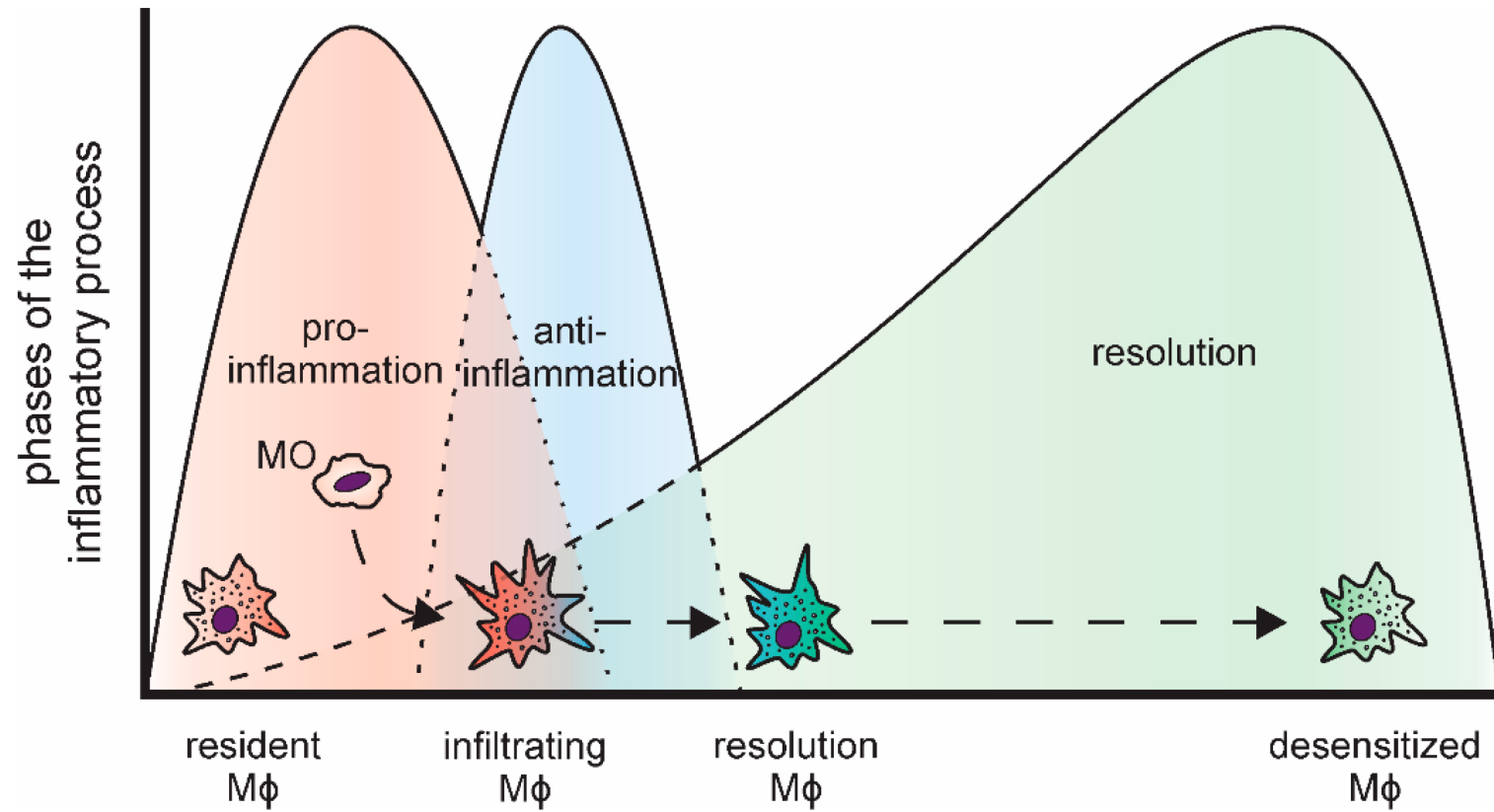
- Defining reaction rules

BioNetGen Actions

- Network generation
- Simulation
- Parameter Scans
- Interfacing with MATLAB

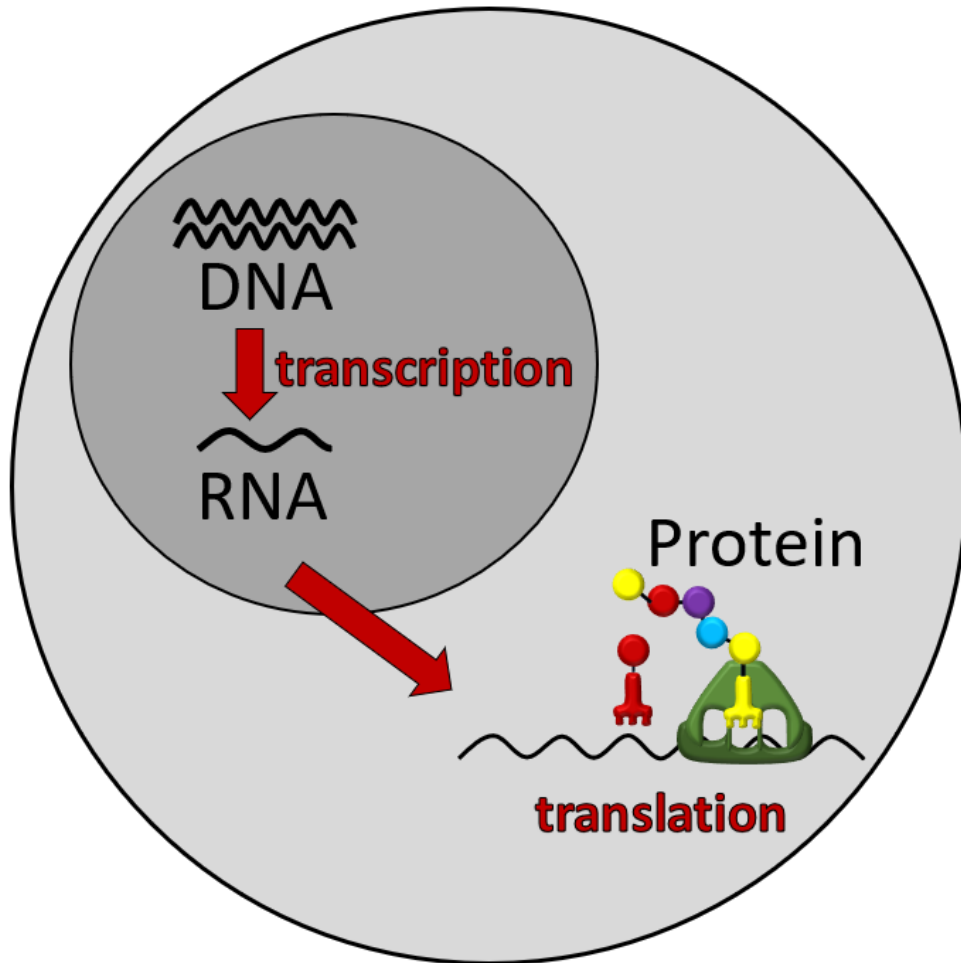


Macrophage behavior depends on extracellular signals



Rappl et al. 2021

Gene expression is determined by transcription factors

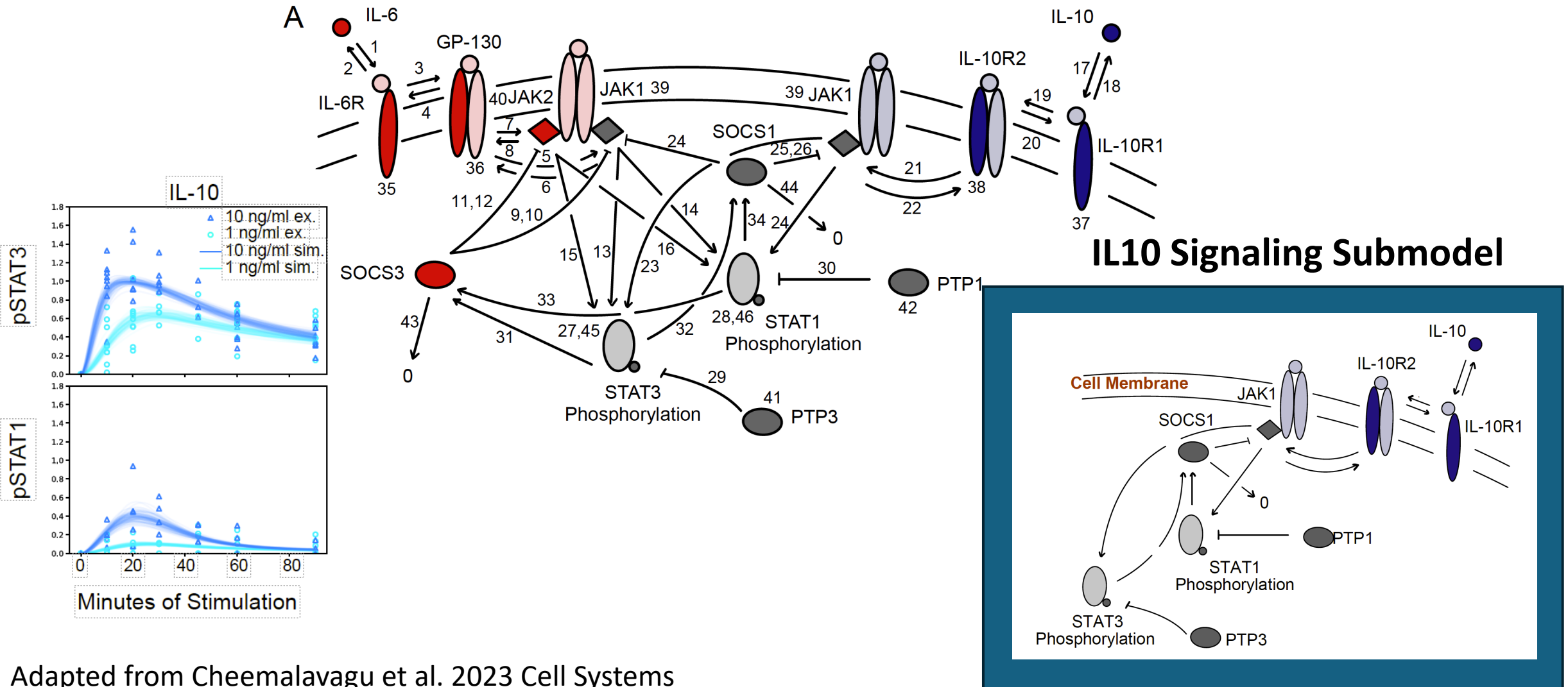


Central Dogma: DNA → RNA → Protein

- **Protein:** Cell behavior and function
- **Which RNA?:** Transcription Factors

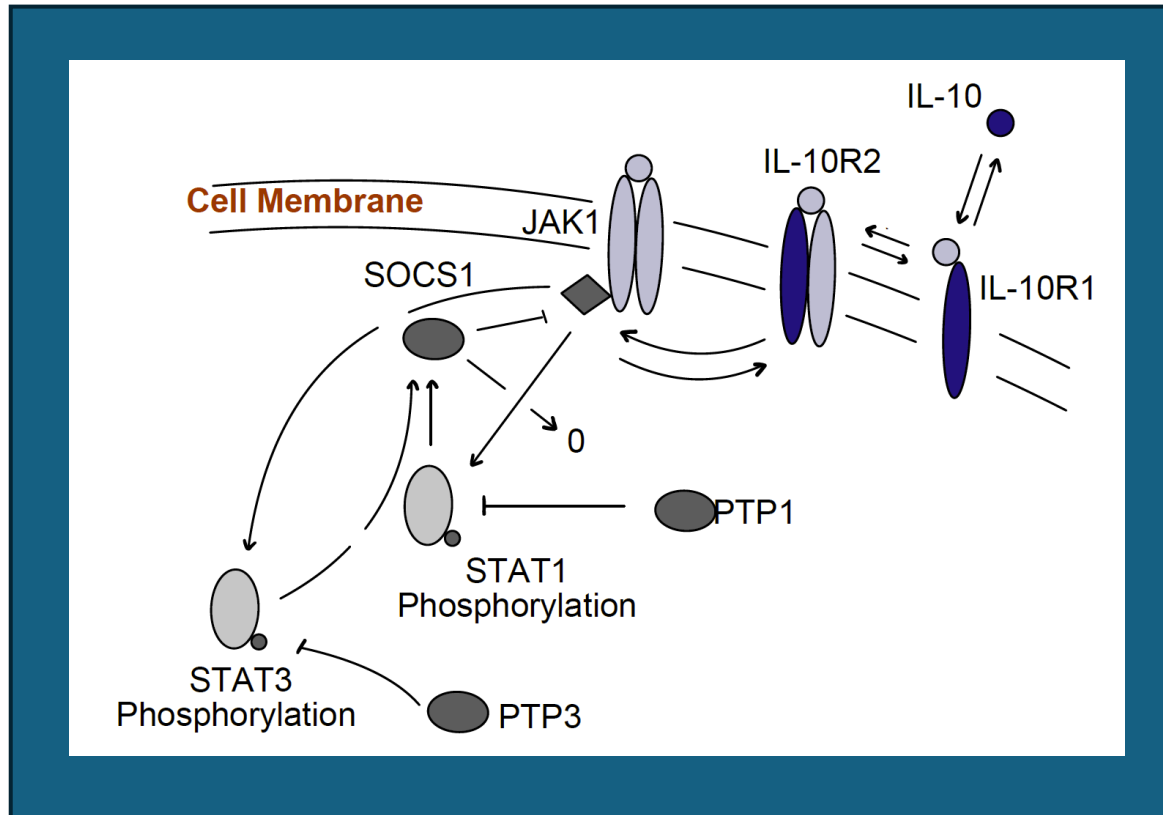
How do extracellular signals influence transcription factor activation?

JAK-STAT signaling shapes macrophage function



Adapted from Cheemalavagu et al. 2023 Cell Systems

STAT Activation Mechanism

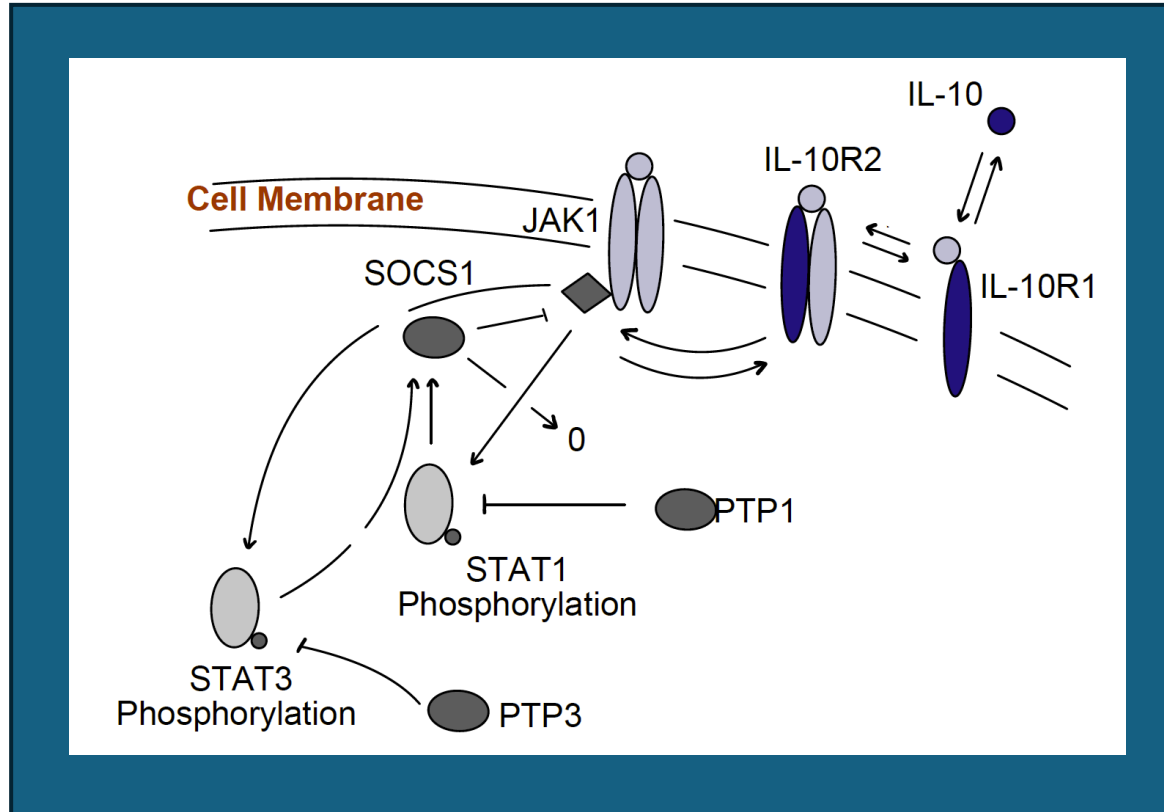


Core Mechanism:

1. IL10 binds receptor
2. Receptor dimerizes
3. Dimer binds JAK1
4. JAK1 activates receptor
5. Active receptor phosphorylates STATs
6. STATs induce SOCS1 production
7. SOCS1 binds JAK1 (inhibition)
8. SOCS decay
9. PTP reverses STAT activation

Mechanism Components

Nine Molecule Types



Adapted from Cheemalavagu et al. 2023 Cell Systems

Molecule	Function
IL10	Signal
IL10R1	Receptor
IL10R2	Receptor
JAK1	Activator
PTP1	Inhibitor
PTP3	Inhibitor
SOCS1	Negative Feedback
STAT1	Observable
STAT3	Observable

BioNetGen Model Construction in VSCode



```
1 # Example_SimpleSTAT.bngl
2 # Molecule Types: 9
3 # Observables: 2
4 # Reaction Rules: 13
5 # Parameters: 28 total
6 #   kinetic constants : 17
7 #   initial conditions: 11
8
9 begin model
10 > begin parameters...
11   end parameters
12
13 > begin molecule types...
14   end molecule types
15
16 > begin seed species...
17   end seed species
18
19 > begin observables...
20   end observables
21
22 > begin reaction rules...
23   end reaction rules
24 end model
25
26 # Actions:
27 # -----
```

Reference File: Example_SimpleSTAT.bngl

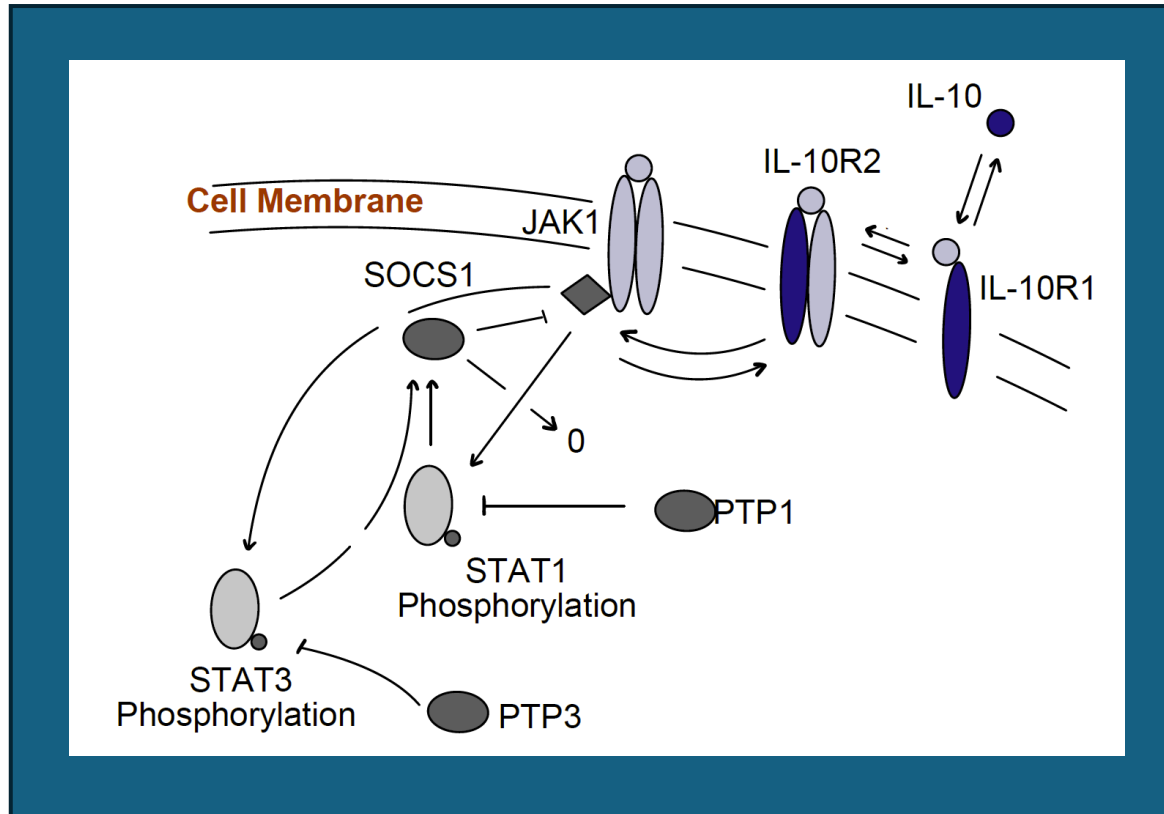
Model Blocks

1. Parameters
2. Molecule Types
3. Seed Species
(Initial Conditions)
4. Observables
5. Reaction Rules

Buttons on Top Right



BioNetGen Model Construction: Molecule Types



Adapted from Cheemalavagu et al. 2023 Cell Systems

```

66 begin molecule types
67
68     IL10(il10r1)
69     IL10R1(l2,il10r2,jak1,stat)
70     IL10R2(il10r1)
71
72     JAK1(rec,socs1)
73
74     SOCS1(jak1)
75
76     PTP3()
77     PTP1()
78
79     STAT3(Y~0~P)
80     STAT1(Y~0~P)
81
82 end molecule types
    
```

[Reference File: Example_SimpleSTAT.bngl](#)

Note: This model does not explicitly describe PTP binding to pSTAT's

BioNetGen Construction: Reaction Rules

```
123 begin reaction rules
124
125     # ===== IL10 Reactions =====
126
127     # ---- (01) IL10 binds to the receptor IL10R1 ----
128     IL10R1(il10,il10r2,jak1,stat) + IL10(il10r1) <-> IL10R1(il10!1,il10r2,jak1,stat).IL10(il10r1!1) il10_il10r1_binding,il10_il10r1_unbinding
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162     # ---- (12) STAT1 induced SOCS1 protein production ----
163     STAT1(Y~P) -> STAT1(Y~P) + SOCS1(jak1) STAT1_SOCS1_ind
164
165     # ---- (13) SOCS1 protein degradation ----
166     SOCS1(jak1) -> 0 SOCS1_degrad
167
168 end reaction rules
```

Reference File: [Example_SimpleSTAT.bngl](#)

BioNetGen Model Construction: Parameter Values

```
10  begin parameters
11
12      # IL-10 constants (8)
13      # -----
14      il10_il10r1_binding 0.200815
15      il10_il10r1_unbinding 0.00000774704
16      il10r1_il10r2_binding 3.15924
17      il10r1_il10r2_unbinding 0.00000361185
18      il10_complex_jak1_binding 1003.25
19      il10_complex_jak1_unbinding 0.0000476335
20      il10_jak1_med_STAT3_act 1.08116
21      il10_jak1_med_STAT1_act 0.0000000197800
22
23      # common constants (4)
24      # -----
25      SOCS1_jak1_binding 9.23454
26      SOCS1_jak1_unbinding 0.00000415259
27      pSTAT3_rec_dissoc 1.77990
28      pSTAT1_rec_dissoc 3.16032
29
30      # STAT parameters (4)
31      # -----
32      PTP_med_STAT3_deact 0.046620
33      PTP_med_STAT1_deact 0.111965
34      STAT3_SOCS1_ind 0.00556053
35      STAT1_SOCS1_ind 11.00256433
36
37      # initial ligand concentrations (1)
38      # -----
39      IL10_0 10
40
41      # initial receptor concentrations (4)
42      # -----
43      IL10R1_0 2.12893
44      IL10R2_0 5889.76
45
46      # initial jak concentrations (1)s
47      # -----
48      JAK1_0 6.83256
49
50      # negative regulators (4)
51      # -----
52      SOCS1_0 0
53
54      PTP3_0 1
55      PTP1_0 1
56
57      SOCS1_degrad 10.2990
58
59      # initial unphosphorylated STAT3 (2)
60      # -----
61      STAT3_0 48.2472
62      STAT1_0 929991
63
64  end parameters
```

Reference File: Example_SimpleSTAT.bngl

BioNetGen Model Construction: Seed Species

```
84 begin seed species
85
86     IL10(il10r1) IL10_0
87     IL10R1(l2,il10r2,jak1,stat) IL10R1_0
88     IL10R2(il10r1) IL10R2_0
89
90     JAK1(rec,socs1) JAK1_0
91
92     SOCS1(jak1) SOCS1_0
93
94     PTP3() PTP3_0
95     PTP1() PTP1_0
96
97     STAT3(Y~0) STAT3_0
98     STAT1(Y~0) STAT1_0
99
100 end seed species
101
```

Seed Species = Initial Conditions

Use Parameter Names!

Reference File: [Example_SimpleSTAT.bngl](#)

BioNetGen Model Construction: Observables

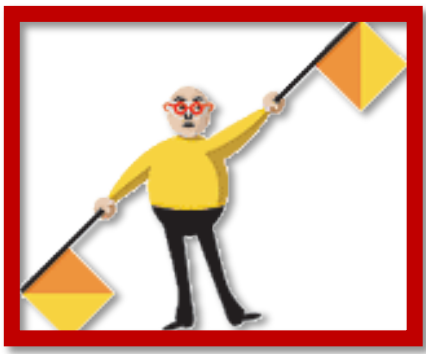
Two types of observables

- **Molecules:** Total quantity of molecule of interest
- **Species:** Total quantity of species containing the molecule of interest

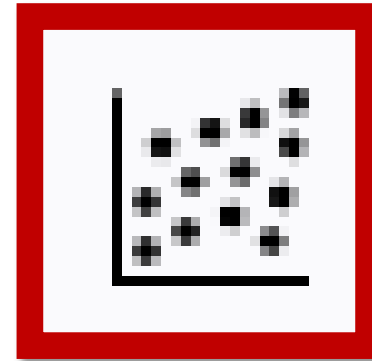
```
102 begin observables
103
104   # pSTAT Observables [!? = Any binding state]: How many molecules in each state
105   # -----
106   # In this system, complexes contain at most one pSTAT molecule, thus the "Molecules" and "Species" definitions produce the same result
107   # To verify this see "groups block" in the .net file
108
109   # Shorthand Definition
110   Molecules total_pSTAT3 STAT3(Y~P!?) #Total quantity of pSTAT3 molecules
111   Molecules total_pSTAT1 STAT1(Y~P!?) #Total quantity of pSTAT1 molecules
112
113   # Longform Definition
114   Molecules total_pSTAT3_explicitdefinition STAT3(Y~P),IL10R1(il10!1,il10r2!2,jak1!3,stat!5).IL10(il10r1!1).IL10R2(il10r1!2).JAK1(rec!3).STAT3(Y~P!5)
115   Molecules total_pSTAT1_explicitdefinition STAT1(Y~P),IL10R1(il10!1,il10r2!2,jak1!3,stat!5).IL10(il10r1!1).IL10R2(il10r1!2).JAK1(rec!3).STAT1(Y~P!5)
116
117   # Shorthand Definition
118   Species total_pSTAT3_species STAT3(Y~P!?) #Total quantity of complexes containing at least one pSTAT3 molecule
119   Species total_pSTAT1_species STAT1(Y~P!?) #Total quantity of complexes containing at least one pSTAT1 molecule
120
121 end observables
```

Reference File: [Example_SimpleSTAT.bngl](#)

Shorthand for any binding state: **?! [Reference .net file for verification]**



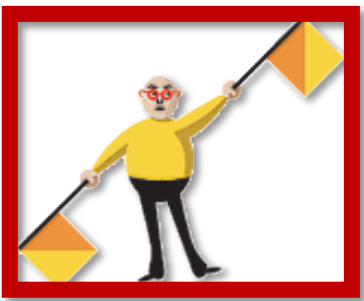
BioNetGen Actions



```
157 end model
158
159 # Actions:
160 # -----
161
162 # For a introduction to BioNetGen actions, see sekar2012rulebased (https://link.springer.com/protocol/10.1007/978-1-61779-833-7\_9), section 6.3.1
163 # Uncomment the desired options below and click "Run BNG" (VScode: stick figure with flags) to proceed
164 #
```

Reference File: Example_SimpleSTAT.bngl

Action	Output File Directory (Example_SimpleSTAT/)
Generate the BioNetGen Network	2026_07_02__11_33_49_generatenetwork_option01 2026_07_02__11_34_17_generatentwork_option02
Visualize a Contact Map	2026_07_02__11_34_35_contactmap_networkoption02
Simulate	2026_07_02__11_35_09_simulate_IL10_10 2026_07_02__11_36_05_simulate_IL10_01
Parameter Scan	2026_07_02__11_37_06_parameterscan_IL10
Construct a MATLAB model (.m)	2026_07_02__11_37_55_writeMfile
Construct a C model (_ccode.c)	2026_07_02__11_38_20_writeMexfile



Action: Generate the BioNetGen Network

The Two Methods for Generating a Network are functionally equivalent

```
171 # Actions:
172 # -----
173
174 # For a introduction to BioNetGen actions, see sekar2012rulebased (https://link.springer.com/protocol/10.1007/978-1-61779-833-7\_9), section 6.3.1
175 # Uncomment the desired options below and click "Run BNG" (VScode: stick figure with flags) to proceed
176 #
177 # (01) Generate the model network, overwriting any existing .net files (output: .net and .log file)
178 # -----
179 # Note: BioNetGen uses the rules above to construct a model. The "generate_network" process only needs to be done once.
180 # The resulting .net file describes the parameters, model species (state variables), and all reactions in the constructed model.
181 # The .log file summarizes the model generation process and the quantity of times each reaction rule is represented in the constructed model.
182 #
183 # Choice between Option 01 and Option 02, below, is user-preference.
184
185 # Option 01: In the .net file, describe the model reactions using reaction rule specifications
186 #generate_network({overwrite=>1,TextReaction=>1});
187
188 # Option 02: In the .net file, describe the model reactions using reaction rule indices
189 #generate_network({overwrite=>1})
190
```

Reference File: [Example_SimpleSTAT.bngl](#)

Output: BioNetGen Network Files

Option 01: Reactions described in words

```
68 begin reactions_text
69   1 IL10R1(il10r2,jak1,l2,stat) + IL10(il10r1) -> IL10(il10r1!1).IL10R1(il10r2,jak1,l2,stat)
70   2 SOCS1(jak1) -> SOCS1_degrad #_R13
71   3 IL10(il10r1!1).IL10R1(il10r2,jak1,l2!1,stat) -> IL10R1(il10r2,jak1,l2,stat).IL10(il10r1!1)
72   4 IL10(il10r1!1).IL10R1(il10r2,jak1,l2!1,stat) + IL10R2(il10r1) -> IL10R1(il10r2,jak1,l2,stat).IL10R2(il10r1!1)
73   5 IL10(il10r1!1).IL10R1(il10r2!2,jak1,l2!1,stat).IL10R2(il10r1!2) -> IL10R1(il10r2!2,jak1,l2!1,stat).IL10R2(il10r1!2)
74   6 IL10(il10r1!1).IL10R1(il10r2!2,jak1,l2!1,stat).IL10R2(il10r1!2) + JAK1 -> IL10R1(il10r2!2,jak1,l2!1,stat).IL10R2(il10r1!2).JAK1
75   7 IL10(il10r1!1).IL10R1(il10r2!2,jak1!3,l2!1,stat).IL10R2(il10r1!2).JAK1 -> IL10R1(il10r2!2,jak1!3,l2!1,stat).IL10R2(il10r1!2).JAK1
76   8 IL10(il10r1!1).IL10R1(il10r2!2,jak1!3,l2!1,stat).IL10R2(il10r1!2).JAK1 -> IL10R1(il10r2!2,jak1!3,l2!1,stat!4).IL10R2(il10r1!2).JAK1
77   9 IL10(il10r1!1).IL10R1(il10r2!2,jak1!3,l2!1,stat).IL10R2(il10r1!2).JAK1 -> IL10R1(il10r2!2,jak1!3,l2!1,stat!4).IL10R2(il10r1!2).JAK1
78   10 IL10(il10r1!1).IL10R1(il10r2!2,jak1!3,l2!1,stat).IL10R2(il10r1!2).JAK1 -> IL10R1(il10r2!2,jak1!3,l2!1,stat!4).IL10R2(il10r1!2).JAK1
79   11 IL10(il10r1!1).IL10R1(il10r2!2,jak1!3,l2!1,stat).IL10R2(il10r1!2).JAK1 -> IL10R1(il10r2!2,jak1!3,l2!1,stat!4).IL10R2(il10r1!2).JAK1
80   12 IL10(il10r1!1).IL10R1(il10r2!2,jak1!3,l2!1,stat!4).IL10R2(il10r1!2).JAK1 -> IL10R1(il10r2!2,jak1!3,l2!1,stat!4).IL10R2(il10r1!2).JAK1
81   13 IL10(il10r1!1).IL10R1(il10r2!2,jak1!3,l2!1,stat!4).IL10R2(il10r1!2).JAK1 -> IL10R1(il10r2!2,jak1!3,l2!1,stat!4).IL10R2(il10r1!2).JAK1
82   14 PTP3() + STAT3(Y~P) -> PTP3() + STAT3(Y~0) PTP_med_STAT3_deact #_R9
83   15 PTP1() + STAT1(Y~P) -> PTP1() + STAT1(Y~0) PTP_med_STAT1_deact #_R10
84   16 STAT3(Y~P) -> STAT3(Y~P) + SOCS1(jak1) STAT3_SOCS1_ind #_R11
85   17 STAT1(Y~P) -> STAT1(Y~P) + SOCS1(jak1) STAT1_SOCS1_ind #_R12
86 end reactions_text
```

Reference file (output):

2026_07_07__13_47_25_GenerateNetwork_option01/
Example_SimpleSTAT.net

Option 02: Reactions described in shorthand

```
49 begin reactions
50   1 1,2 10 il10_il10r1_binding #_R1
51   2 5 0 SOCS1_degrad #_R13
52   3 10 1,2 il10_il10r1_unbinding #_reverse__R1
53   4 3,10 11 il10r1_il10r2_binding #_R2
54   5 11 3,10 il10r1_il10r2_unbinding #_reverse__R2
55   6 4,11 12 il10_complex_jak1_binding #_R3
56   7 12 4,11 il10_complex_jak1_unbinding #_reverse__R3
57   8 5,12 13 SOCS1_jak1_binding #_R4
58   9 8,12 14 il10_jak1_med_STAT3_act #_R5
59   10 9,12 15 il10_jak1_med_STAT1_act #_R7
60   11 13 5,12 SOCS1_jak1_unbinding #_reverse__R4
61   12 14 12,16 pSTAT3_rec_dissoc #_R6
62   13 15 12,17 pSTAT1_rec_dissoc #_R8
63   14 6,16 6,8 PTP_med_STAT3_deact #_R9
64   15 7,17 7,9 PTP_med_STAT1_deact #_R10
65   16 16 5,16 STAT3_SOCS1_ind #_R11
66   17 17 5,17 STAT1_SOCS1_ind #_R12
67 end reactions
```

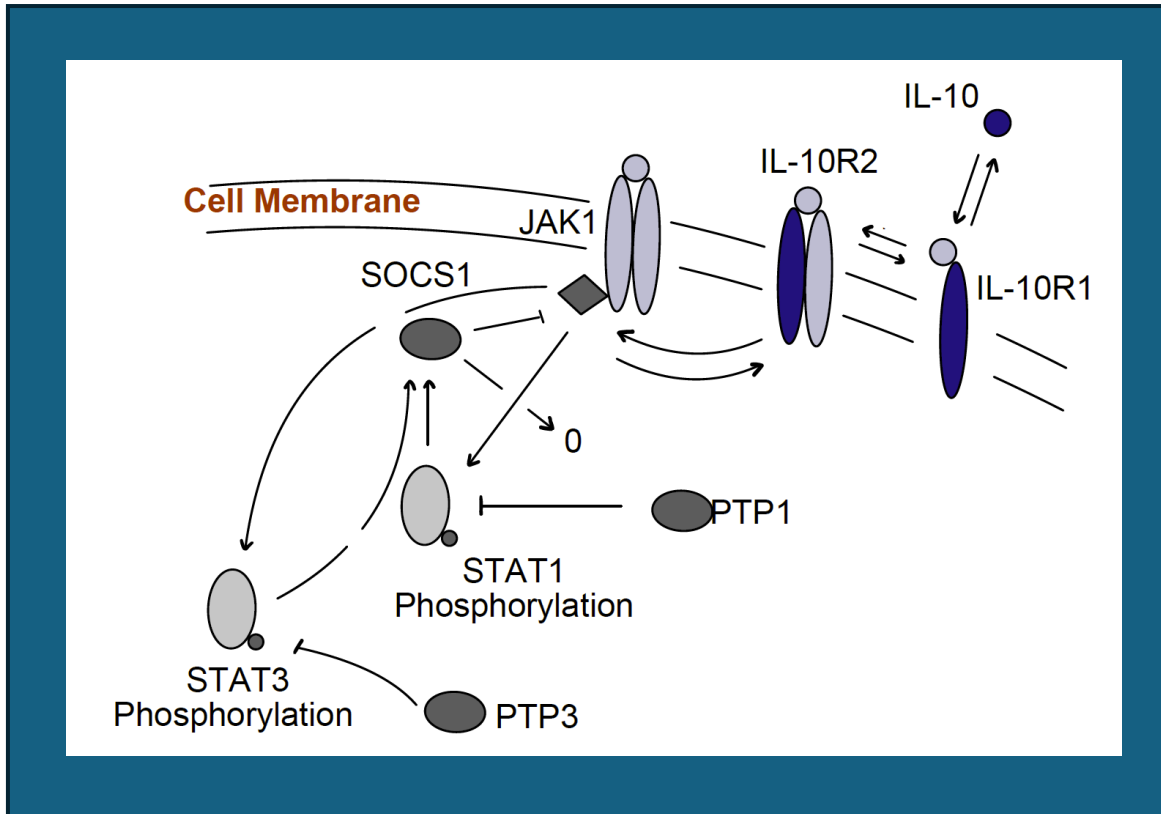
Reference file (output):

2026_07_07__13_48_17_GenerateNetwork_option02/
Example_SimpleSTAT.net

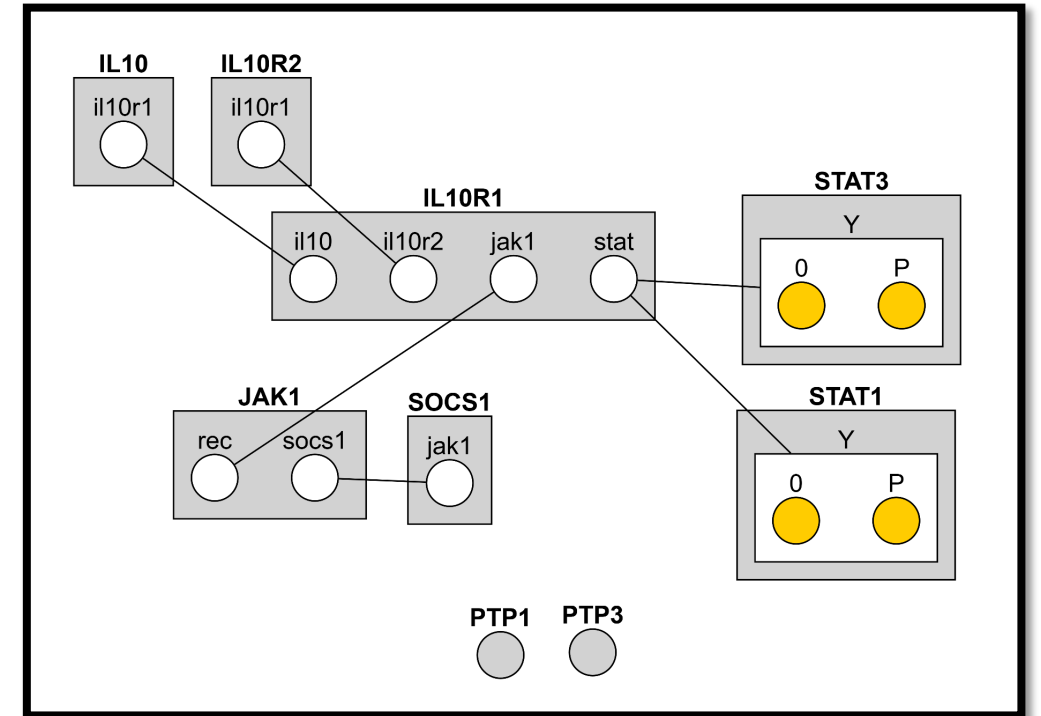
Note: “Groups block” describes observables

Action: Contact Map Construction

Model Diagrams vs. Contact Maps



Model Diagram: Describes mechanism



Contact Map: Describes binding connectivity

*This model does not explicitly describe PTP binding to pSTATs.

Action: Contact Map Construction

Step 0

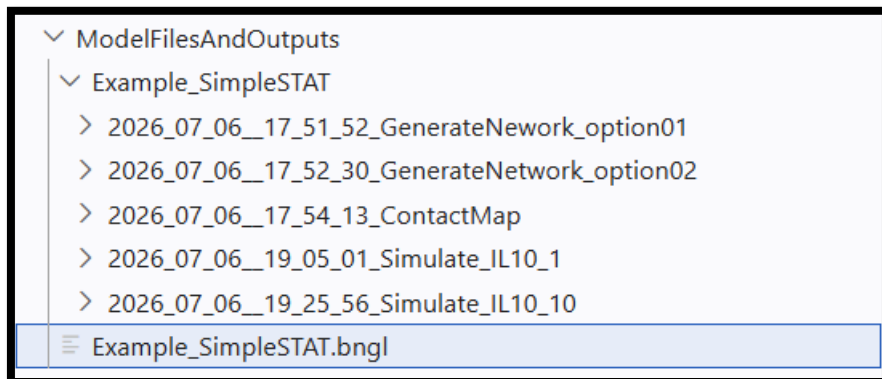
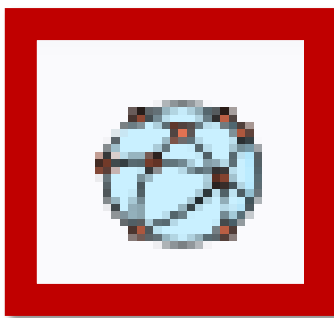
Uncomment
generate_network

```
# Option 01: In the .net file, describe the model reactions using reaction rule indices  
generate_network({overwrite=>1})
```

Example_SimpleSTAT.bngl

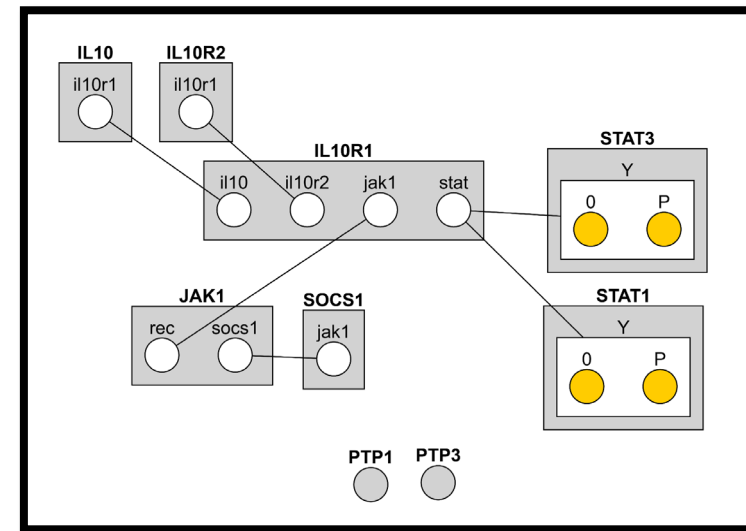
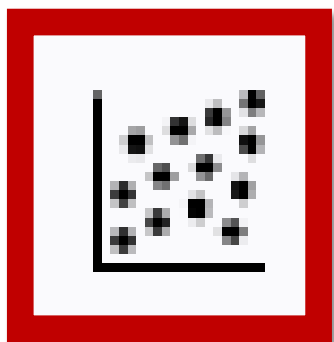
Step 1

Generate
contactmap file



Step 2

Visualize
contact map



Step 3 Manually rearrange as desired

Reference files: 2026_07_07_13_48_59_ContactMap

Action: Simulate



```
179 # (02) Simulate the model using the default parameters and seed values
180 # -----
181 # Note: This creates the following file types:
182 #   - .net : See description below
183 #   - .log : See description below
184 #   - .cdat : Species (state variable) trajectory values
185 #   - .gdat : Observable trajectory values
186
187 simulate({method=>"ode",t_start=>0, t_end=>90,n_steps=>91,print_functions=>1})
188
```

Reference File: Example_SimpleSTAT.bngl

Syntax to generate timecourse data:

```
simulate({method=>"ode",t_start=>0, t_end=>90,n_steps=>91,print_functions=>1})
```

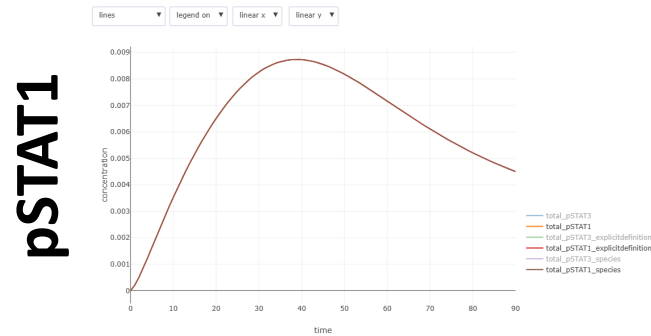
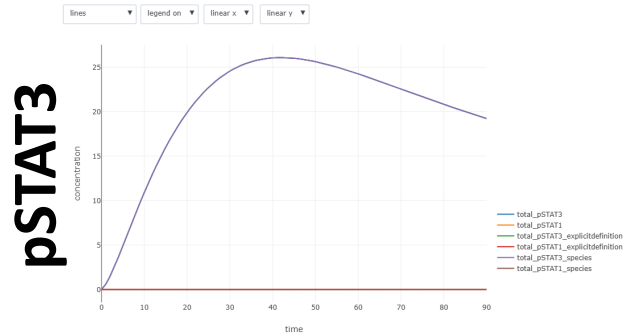
Specify initial conditions in the parameters block:

```
37 # initial ligand concentrations (1)
38 # -----
39 IL10_0 10
```

Reference File: Example_SimpleSTAT.bngl

Simulation Output Files

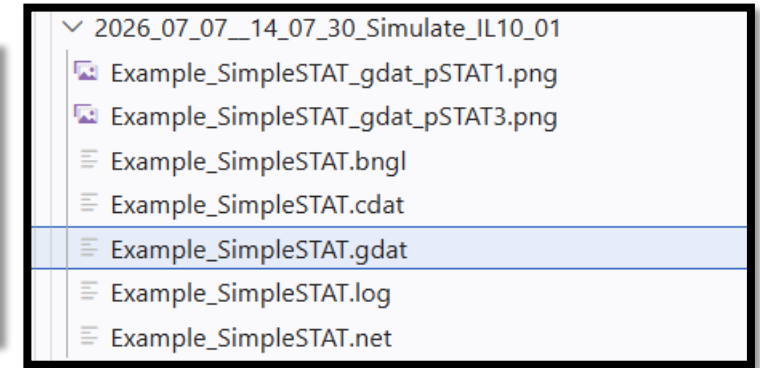
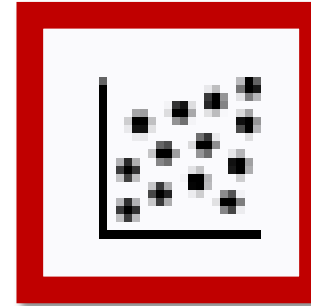
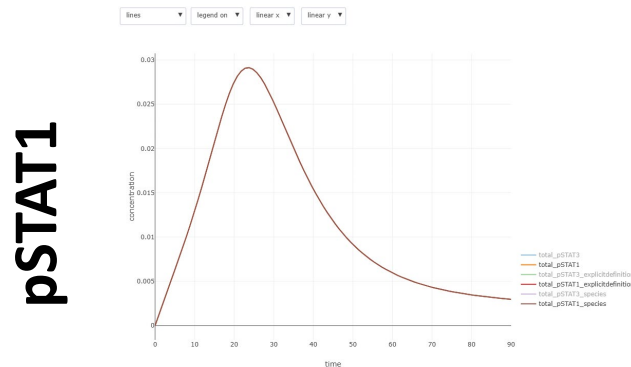
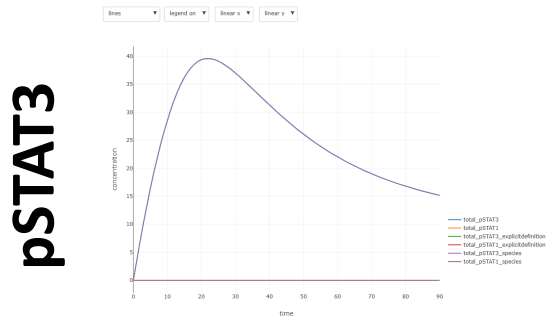
Case IL10: 1 unit



Time (min)

Time (min)

Case IL10: 10 units



VSCode File Tree

Output:

- .gdat: Observables
- .cdat: All Species

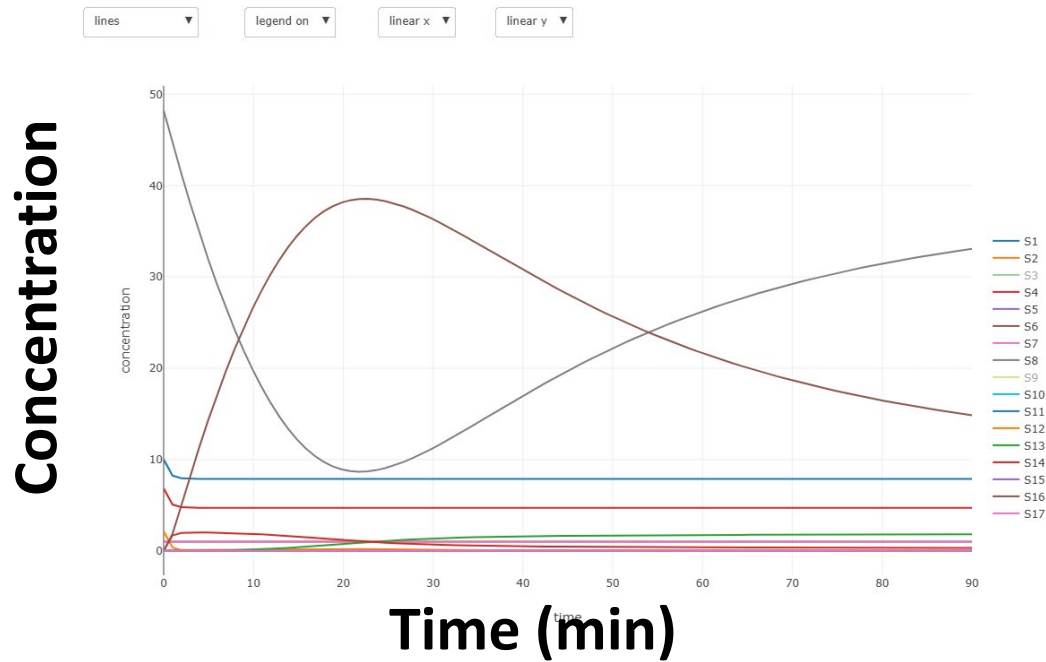
Reference files (output):

2026_07_07__14_07_30_Simulate_IL10_01

2026_07_07__23_08_13_Simulate_IL10_10

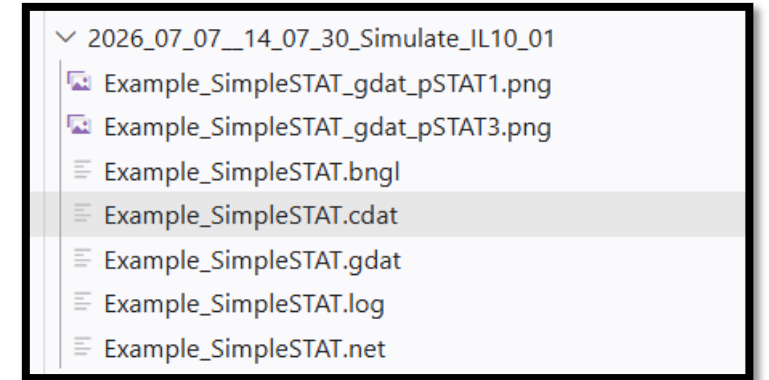
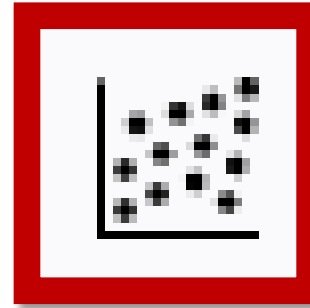
Action: Simulate

Case IL10: 10 units



Notes:

- S3 and S9 are excluded from this figure for clarity
- See .net file for species descriptions



VSCode File Tree

Output:

- .gdat: Observables
- .cdat: All Species

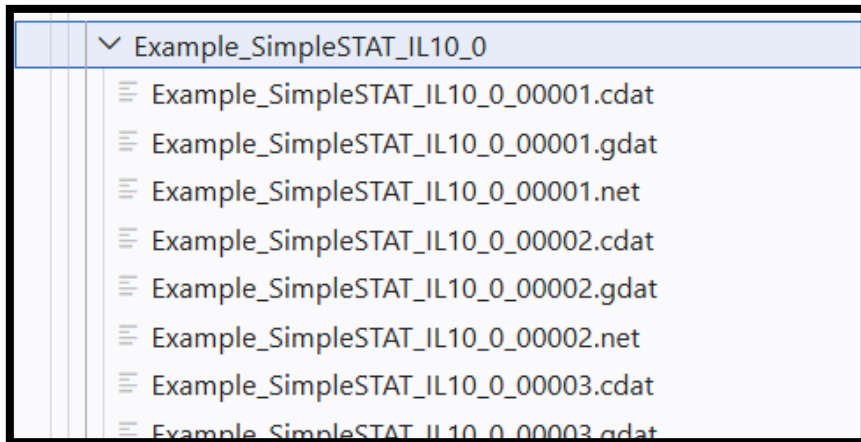
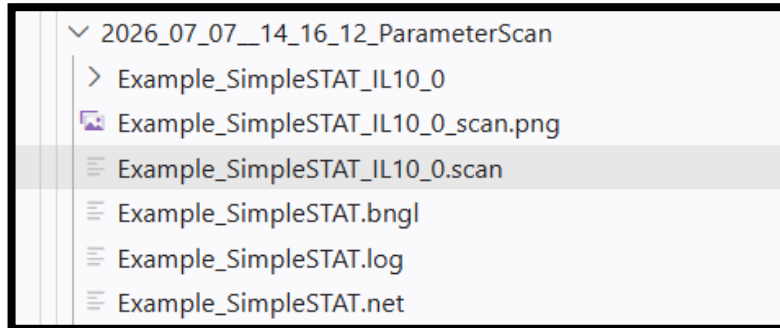
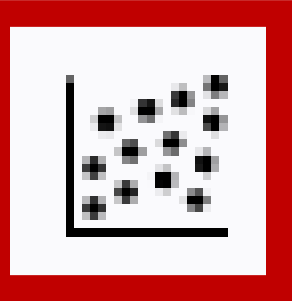
Reference files (output):

2026_07_07__14_07_30_Simulate_IL10_01
2026_07_07__23_08_13_Simulate_IL10_10

Action: Parameter Sweep

```
202 # (03) Parameter scan
203 #-----
204 parameter_scan({method=>"ode", t_end=>25, parameter=>"IL10_0", par_min=>0, par_max=>10, n_scan_pts=>100})
205
```

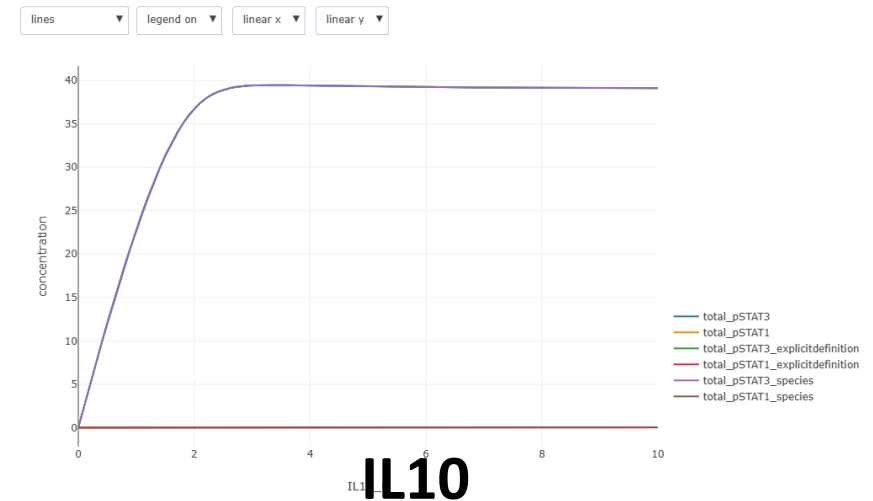
Reference File: Example_SimpleSTAT.bngl



VSCoDe File Tree

pSTAT3 conc. at 25 min for increasing IL10 stimulus

pSTAT3



Reference files (output):

2026_07_07_14_16_12_ParameterScan

Action: Constructing a MATLAB model

MATLAB Simulation

- Step 1: Create a .m file

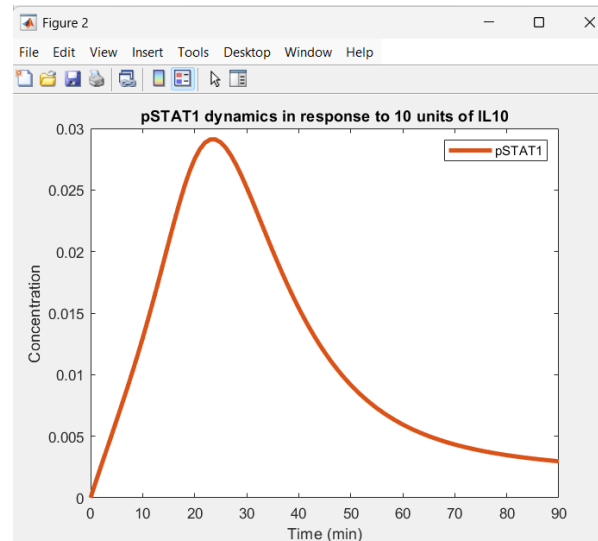
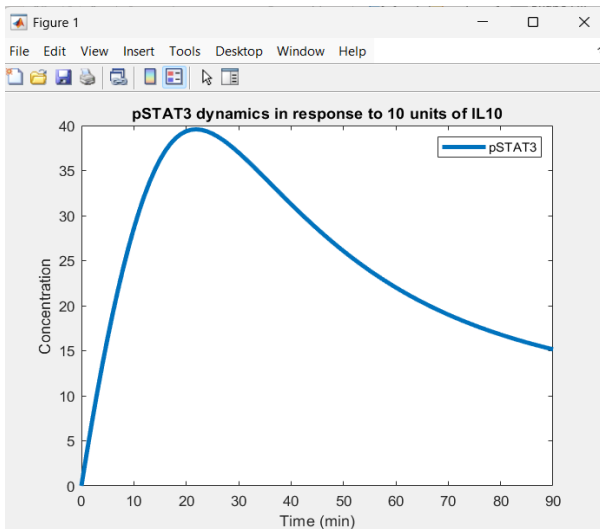
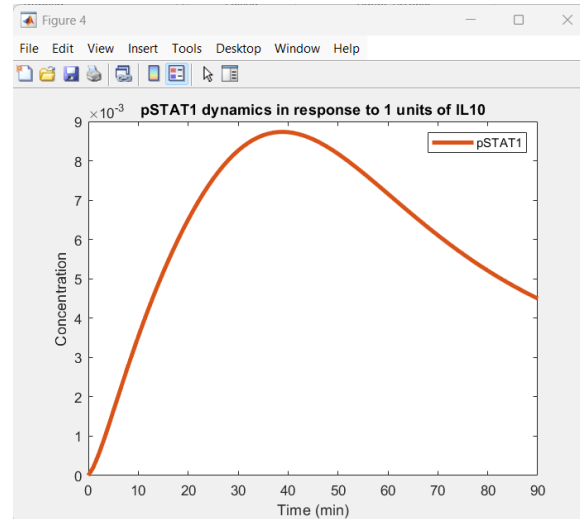
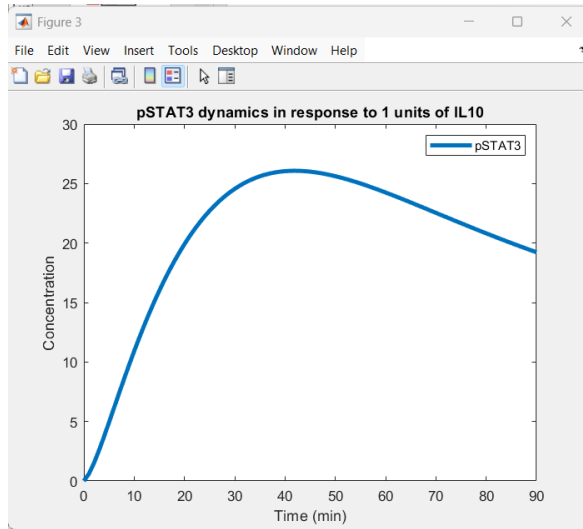
Reference Files (output): 2026_07_07__14_23_27_GenerateMFile

```
206 # (03) Generate a .m file
207 #-----
208 # Recommended: Begin by simulating the system to verify that the model is specified correctly before generating .m and *_cnode.c files.
209 # Note: Requires simultaneous use of "generate_network", uncomment the desired option above when generating a .m file. Demonstration used option 01.
210 #
211 # writeMfile creates the following output:
212 # - .m      : Uses .net file to construct a .m model file that takes a vector of initial conditions, parameters, and timepoints,
213 #            passes them into the model equations and integrates with ode15s.
214
215 writeMfile({t_start=>0,t_end=>90,n_steps=>91,atol=>1e-10})
```

Reference File: Example_SimpleSTAT.bngl

- Step 2: Simulate with **Scripts/Msimulate/Simulate_Example_SimpleSTAT.m**
 - Option 01: Autogenerate observable plots
 - Option 02: Custom plots

Interfacing with MATLAB: Simulation



```
% Simulate and auto generate plot (Uses ode15s)
% [err, timepoints, species_out, observables_out] = Example_SimpleSTAT( timepoints, species_init, parameters, 0);

% Custom Plot (uses ode15s)
species_init(01) = 10; % IL10(il10r1) IL10_0
[err, timepoints, species_out, observables_out] = Example_SimpleSTAT( timepoints, species_init, parameters, 1);
```

Reference File:

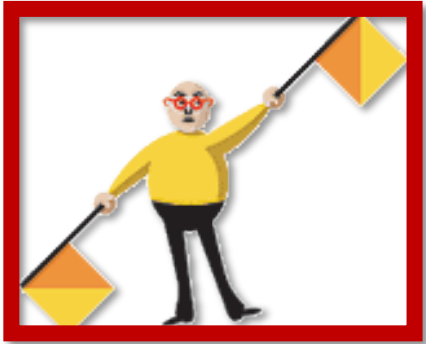
Scripts/MSimulate/Simulate_Example_SimpleSTAT

Default Integration: ode15s

Provided script:

- Auto-generation of figures using BioNetGen .m file
- Custom generation of figures (shown)

Interfacing with MATLAB: Ccode/mex



```
218 # (04) Generate a *_ccode.c file
219 # -----
220 # Note: Requires simultaneous use of "generate_network" and "writeMfile", uncomment the desired options above when generating a *_ccode.c file.
221 #
222 # WriteMfile creates the following output:
223 # -----
224 # - *_ccode.c : Uses the .m file to construct a .c model file.
225 #
226 # Note: Annotations at the top of the *_ccode.c file provide instructions for compiling the .mex file
227
228 writeMexfile({t_start=>0,t_end=>90,n_steps=>91,atol=>1e-10})
```

Reference File: Example_SimpleSTAT.bngl

Uncomment:

- generate_network
- writeMfile
- writeMexfile

Reference Files (output): 2026_07_07__14_35_04_GenerateMexFile

Interfacing with MATLAB: Ccode/mex

Provided Scripts:

MexSimulation/

- MexCompilationTemplate_Linux
- Simulate_Example_SimpleSTAT_ccode

Steps to generate custom figures:

1. Construct a _ccode.c file using Example_SimpleSTAT.bngl (previous slide)
2. Customize MexCompilationTemplat_Linux.m and run to compile
3. Run Simulate_Example_SimpleSTAT_ccode.c to simulate and graph

Outline



Jim Faeder

- Interfacing with Python and Model Analysis

Alex Dibiasi

- Network-based vs. Network Free Simulation

Achyudhan Kutuva

- BNG Playground

Additional Resources

<https://bionetgen.org/resources>

<https://bionetgen.org/tutorial>